Development Plan
Flexin

Team 3, Flexin
Arjun Karthik
Shao (Daniel) Hu
Noah Goldschmied
Peter Deforest
Daniel Franze-Da Silva

Table 1: Revision History

Date	**Developer(s)**	**Change**
September 26th, 2026	Arjun	Added workflow plan and project decomposition and scheduling
September 26th, 2026	Daniel F.	Added team member roles, team communication plan, team meeting plan and team charter (external goals, attendance, acceptable excuses, in case of emergency, quality and attitude)
September 26th, 2026	Daniel H.	Added use of AI/LLMs: code development, documentation development and verification of LLM output
September 26th, 2026	Noah	Added proof of concept demonstration plan
September 27th, 2026	Daniel F.	Added team charter (stay on track, decision making and team building); grammar fixes to team charter and team meeting plan
September 27th, 2026	Arjun	Fixed typo in project scheduling
September 27th, 2026	Noah	Added another risk to proof of concept demonstration plan
September 27th, 2026	Team	Call to review documents
September 27th, 2026	Daniel F.	Added introduction, confidential information, IP to protect, and copyright license
September 27th, 2026	Peter D.	Added Expected Technologies information, Added Coding Standards information
September 28th, 2026	Arjun	Fixed and added entries in project scheduling
September 28th, 2026	Peter D.	Added AI-Generated content verification

1 Introduction

This document outlines the development plan for the fitness and financial learning app designed to teach users the benefits of incremental investments in both your health and finances. This document includes legal information, internal team plans and roles throughout the project, workflow plans and deadline schedules, proof of concept plans, and technologies to be utilized for development.

2 Confidential Information?

There is no confidential information to protect from industry partners.
Note: This app will deal with personal user data that may be deemed confidential. This data will be secured and more thoroughly disclosed in the requirements documentation. This data will be accumulated and is not coming from an industry partner.

3 IP to Protect

Any software and documentation associated with this project are protected by copyright (see copyright license section below). The copyright license prohibits any individuals or groups the rights to use, modify, or copy the software or its documentation without explicit permission granted by the copyright holders. Any unauhthorized use of the project's artifacts will result in legal action.

4 Copyright License

The team will be adopting a proprietary license that reserves all rights to the copyright holders. The license can be found in the LICENSE file.

5 Team Meeting Plan

When a lecture is held (where the professor is delivering material - primarily until October 1st, and a few times throughout the winter 2027 semester), the team will spend 5-10 minutes, post-lecture, informally meeting to touch base on the progress made, and any roadblocks that may need to be addressed. These informal meetings will act as 'stand-up' style meetings where the team can hold each other accountable for the continuous work they have completed and identify any roadblocks in the development early so that they can be addressed with the help of other team members ahead of project deadlines. On days where there is no lecture, due to this time already being reserved in our calendars, the team will hold more formal meetings during these time slots to discuss in greater depth progress made, resolving any road blocks, and action items for each team member ahead of next meeting. These meetings will be held in-person on campus or virtually over Microsoft Teams. The exact location will be

discussed and determined by the team on the Sunday at the start of each week, in order to best accommodate the schedules of all team members and to account for necessary accomodations for commuters. The scheduled lecture time used for meetings will occur every week on Mondays, Wednesdays, and Thursdays from 5:30-6:20 PM.

During scheduled tutorial time (weekly on Wednesdays from 2:30-4:20 PM in the fall semester, and Tuesdays from 2:30-4:20 PM in the winter semester), the team will meet in person. The first 5-10 minutes will be spent preparing and reviewing the agenda for our discussion with the TA, 20 minutes will be dedicated to the discussion with the TA to receive feedback and ask any pending questions (note: our TA is also our supervisor, thus this time will also be served to address any requirements based questions), and then any remaining time will be dedicated to development time where the team will work on dicussing and/or implementing critical logic that requires extra attention and alignment on.

In addition, the team will hold weekly meetings outside of class time (lectures and tutorials) every Tuesday from 10:00-11:00 AM. These meetings will be held virtually over Microsoft Teams. The purpose of these meetings will be to discuss upcoming deliverables and planning for the week ahead (i.e. this may include assigning action items for a project artifact).

All meetings are subject to rescheduling and/or cancellation based on discussion where all team members are in agreement, and will be communicated to all team members at least 24 hours in advance. Meeting rescheduling and/or cancellation may be due to holidays, McMaster University shutdown days, or group consensus to postpone a meeting (**excpetion**: tutorial meetings cannot be rescheduled and/or cancelled due to group consenus as meeting with the TA is a mandatory and beneficial part of the development feedback process).

For each of the meetings mentioned above, an agenda will be determined at the end of the previous meeting. Each meeting will closely follow this agenda, leaving time at the end for any questions to be addressed. The roles for each meeting will remain consistent, as to maintain efficiency in the meetings. Each meeting Noah will serve as the meeting chair, responsible for leading the meeting, Daniel F. will serve as the notetaker, responsible for capturing talking points and recording action items, and Arjun and Daniel F. will be responsible for capturing meeting attendance as issues on GitHub. The meeting template (MeetingMinutesTemplate.pdf) will be used to capture meeting minutes, agendas, action items, and other details about each meeting. These meeting records will be stored on the shared Microsoft Teams channel for the team for later reference.

6 Team Communication Plan

The following outlines the communication tools that will be used throughout the project:

- **GitHub:** a main repository will be used for version control and issue tracking. As well, a separate GitHub project will be used for project

management, where issues will be tracked using a Kanban board with 4 states: To Do, In Progress, In Review, and Done.

- **Microsoft Teams:** Microsoft Teams will be used as the main source of communication between team members. A Microsoft Teams group chat will be used for adhoc messaging and communication. A Microsoft Teams Channel will be used for conducting virtual team meetings, storing any shared files that do not belong in the code repository, and posting communication regarding pull requests and their reviews.
- **Email:** our McMaster emails will be used for communication with our supervisor/TA and professors.

7 Team Member Roles

All team members will be responsible for contributing to the development of the project (this includes coding, documentation, testing, reviewing, etc.). In addition to this contribution, each team member will be assigned the following roles (roles can be modified if necessary):

Table 2: Team Member Role Assignment

Team Member	**Role(s)**	**Responsibilities**
Daniel Franze-Da Silva	Scribe	Take thorough notes during meetings
	UI/UX Design Lead	In charge of ensuring the UI and UX meet the high standards of the end users
Noah Goldschmied	Team Liason	Act as the main point of contact for the team when communicating with external individuals relevant to the project (i.e. TAs, supervisors, professors, potential partnering companies, etc.)
	Chairman	Lead team meetings, set the agenda for upcoming meetings and guide discussions
Arjun Karthik	Github Manager	In charge of managing the GitHub repository, managing and creating issues, and ensuring that the GitHub project is organized and complete
	CI/CD Lead	In charge of ensuring that the CI/CD pipeline is set up and functioning properly, and that all team members are adhering to the regulations put into place (i.e. branching, pull requests and approvals, proper naming conventions, etc.)
Peter Deforest	Testing Lead	In charge of ensuring that proper tests are conducted, regression testing is upheld when new logic is introduced, and that the testing process is properly integrated into the development
Shao (Daniel) Hu	Lead Developer	In charge of leading the development efforts and technical design, and helping teammates overcome technical roadblocks

8 Workflow Plan

The project workflow will be managed using Git, with our code and documentation hosted in our GitHub repository. Our `main` branch will serve as the primary codebase and will be the branch used for product deployments. To make contributions, developers are required to create branches directly from `main` and commit any initial changes to their respective branch.

8.1 Branching Strategy

We will utilize the following branch categories:

- **feat (feature):** For development on a new feature.
- **bugfix:** For addressing and fixing identified issues within the codebase.
- **chore:** For code changes that support later development, such as refactoring or clean-up work.

Additionally, each branch must have a descriptive name that clearly indicates which issue in the GitHub project is being addressed.

8.2 Issue Tracking and Project Management

Issues will be categorized based on urgency, from **P0** (highest priority) to **P2** (lowest priority), ensuring that critical tasks are addressed promptly.

The GitHub project board has been configured with the following setup: **Backlog** → **In Progress** → **Review** → **Closed**. As development progresses, it is the developer's responsibility to keep their assigned issues updated to maintain team visibility on project status.

8.3 Pull Requests and CI/CD Pipeline

As part of the new code added to a branch, developers are required to implement test cases to ensure adequate code coverage. Once code is committed, a Pull Request (PR) must be raised to merge the branch into `main`.

Our Continuous Integration/Continuous Deployment (CI/CD) pipeline will enforce the following checks before a merge is permitted:

- Automated testing and deployment checks.
- Linter execution to ensure code meets formatting and quality standards.
- Verification that the feature branch is up-to-date with `main`.

8.4 Code Review Process

In addition to passing all CI/CD checks, each PR requires a manual review and approval from at least one other group member. The PR comments section will be utilized to trace the iterative process of feedback and the subsequent code changes made to address said feedback.

8.5 Pull Request Template Requirements

Each PR must be directly linked to its corresponding issue in the GitHub Project board to maintain traceability. Furthermore, all PRs must adhere to a standardized template that includes:

- A comprehensive summary of the changes made.
- Sample test cases and step-by-step instructions for the reviewer to verify the code.
- Screenshots of the updated UI (if the PR includes user interface modifications).

9 Project Decomposition and Scheduling

For aiding in documenting and tracing our project deliverables, we will be making use of two GitHub projects that will handle our code and our documentation deliverables. They are linked below:

- **Code Project:** https://github.com/users/arjunkarthik123/projects/4/views/1
- **Documentation Project:** https://github.com/users/arjunkarthik123/projects/3

Both projects are split into four sections: Backlog, In Progress, In Review, and Done. As the project progresses, we will come up with items that need to be addressed for both our documentation and later on our codebase. Whenever any team member thinks of an item that needs to be worked on, it will be added to the backlog of the respective projects. Depending on what the team decides, someone will be assigned the new issue, and whenever the person works on it, it will be under the In Progress section. In order to complete the deliverable, it first needs to be reviewed, hence the In Review section. For either the code or documentation, a pull request will be created and linked to the respective issue being addressed, and only when it gets approved will the issue move to the Done section.

If the issues seem to be larger than anticipated, we will split up the issue using sub-issues and have them assigned accordingly. This is to ensure no single issue is too big to handle by one person alone.

Table 3 provides a high-level overview of the key deliverables throughout the academic year (please note that these items were taken directly from the course schedule in the course outline):

Table 3: Key Deliverables Schedule

Term	**Week**	**Date**	**Tasks / Deliverables**
Term 1	Week 2	Week of Sep 14, 2026	**Deliverable:** Deadline for teams formation, projects selection (end of week)
Term 1	Week 3	Week of Sep 21, 2026	**Deliverable:** Biosheets due to Avenue dropbox **TA Meeting:** Review of prob state, PoC plan, dev plan and team charter
Term 1	Week 4	Week of Sep 28, 2026	**Deliverable:** Problem Statement, Proof of Concept Plan, Development Plan, Team Charter Rev 0 **Peer Review:** Prob State + PoC plan + Dev Plan + Team Charter peer review
Term 1	Week 5	Week of Oct 05, 2026	**TA Meeting:** Review SRS+Haz Analysis **User Interview Questions:** Finalize the interview questions needed for the user interview
Term 1	Week 6	Week of Oct 19, 2026	**Deliverable:** SRS + Haz Analysis Rev 0 **Peer Review:** SRS + Haz Analysis peer review **Interviews Complete:** Have all compiled interviews which can be used to derive requirements for SRS **Market Decision:** Have the preliminary market engine setup that users can invest their virtual currency into for the PoC

Continued on next page

Table 3: Key Deliverables Schedule (Continued)

Term	Week	Date	Tasks / Deliverables
Term 1	Week 7	Week of Oct 26, 2026	**TA Meeting:** Review VnV Plan
Term 1	Week 8	Week of Nov 02, 2026	**Deliverable:** VnV Plan Rev 0 **Peer Review:** VnV Plan peer review
Term 1	Week 9	Week of Nov 09, 2026	**TA Meeting:** Review Des Doc Rev-1 and CD-Fall **Currency Decision:** Define the logic of virtual currency, how workouts translate to in game currency to be used
Term 1	Week 10	Week of Nov 16, 2026	**Deliverable:** Des Doc Rev -1 and CD-Fall **Peer Review:** Des Doc Rev-1 Peer Review
Term 1	Week 11	Week of Nov 23, 2026	**Presentation/Demo:** Proof of Concept Demos
Term 1	Week 12	Week of Nov 30, 2026	**Presentation/Demo:** Proof of Concept Demos
Term 1	Week 13	Week of Dec 07, 2026	**Presentation/Demo:** Code review interviews
Term 2	Week 15	Week of Jan 11, 2027	**TA Meeting:** Review of Des Doc

Continued on next page

Table 3: Key Deliverables Schedule (Continued)

Term	Week	Date	Tasks / Deliverables
Term 2	Week 16	Week of Jan 18, 2027	**Deliverable:** Design Document Rev 0 **Peer Review:** Module Implementation due **Code Review:** Decide the key backend software architecture for all the different markets we want to simulate and how the frontend information gets processed **Code Review:** Decide the frontend software architecture for all the different screens of the application, how to manage the flow of information
Term 2	Week 17	Week of Jan 25, 2027	**Peer Review:** Des Document peer review (module implementation reflection) **User Testing:** By this time, have user testing a demo build and get feedback of user experience and usage **Business Plan:** Draft initial business plan ideas and consider potential monetization strategies
Term 2	Week 18	Week of Feb 01, 2027	**Presentation/Demo:** Rev 0 Demo
Term 2	Week 19	Week of Feb 08, 2027	**Presentation/Demo:** Rev 0 Demo
Term 2	Week 20	Week of Feb 22, 2027	**Presentation/Demo:** Code review interviews

Continued on next page

Table 3: Key Deliverables Schedule (Continued)

Term	Week	Date	Tasks / Deliverables
Term 2	Week 21	Week of Mar 01, 2027	**TA Meeting:** Review VnV Report+CD-Winter **User Testing:** Redo user testing prior to the final demo to make any last minute changes if possible **Business Plan:** Finalize monetization strategy for the business plan after gaining user feedback on how the strategy would impact user experience
Term 2	Week 22	Week of Mar 08, 2027	**Deliverable:** VnV Report + CD-Winter **Peer Review:** VnV Report + CD-Winter peer review
Term 2	Week 23	Week of Mar 15, 2027	**Presentation/Demo:** Final Demos
Term 2	Week 25	Week of Mar 29, 2027	**Deliverable:** EXPO poster pdf and video due; Final Documentation (Rev 1)
Term 2	Week 26	Week of Apr 05, 2027	**Presentation/Demo:** Capstone EXPO

10 Proof of Concept Demonstration Plan

At the time of the proof of concept, the application will be able to perform the below actions:

- Allow users to set fitness and financial goals
- Integrate with fitness technologies to accurately track users' activities
- Award users virtual currency based on their activity in an accurate manner
- Allow users to invest said virtual currency in a financial market
- Provide the user with tooltips and terminology that will help them succeed in real world investing

The main risks that could hinder the success of the proof of concept are:

1. An inability to properly gamify or incentivize compounding habits by rewarding healthy long-term thinking

    - To address this risk, the team will be conducting a design thinking exercise. This will help with the development of these incentives to make sure that this risk is minimized, as it is critical to the success of the project. To prove that this risk is handled, the PoC will implement the incentives discovered by the design thinking exercise to demonstrate that the core thesis of the product is functional.

2. Improper workout verification leading to any of the following:

    (a) Rewarding not enough virtual currency

    (b) Rewarding too much virtual currency

    (c) Users exploiting the tracking system

    - This risk stems from an improper calibration of both the inputs from workout technology, and an improper weighting of the currency system. R&D will be required to fine tune this system and remove this risk

3. Users being unable to invest the virtual currency that they earn

4. Failure to integrate with physical fitness technologies

5. Improper guardrails and teaching points in place

If the application is unable to complete the above steps, the team may need to re-analyze the scope of the project. The single most important risk to meet is the gamification and incentivization of compounding habits. All other risks are secondary, and the PoC should prioritize the development of the incentives. In the case that either workouts or investments are being improperly tracked, the team will look into simplifying the system, while keeping the incentives intact.

11 Expected Technology

11.1 Languages and Frameworks

The application will be developed using the frontend React Native framework (https://reactnative.dev/) using Typescript (https://www.typescriptlang.org/) as the coding language. Typescript is a common programming language for frontend development with many tools to help the development process. Figma (https://www.figma.com/) mockups will also be used to help the development team create UI concepts and examples for testing and concept design.
The backend systems will consist of an SQL relational database through a

MySQL server (https://www.mysql.com/). A relational database will allow the development team to keep more concise and organized records to reduce search and loading times for users. We will use the Java Spring.io framework (https://spring.io/) to develop our API. The API will provide the ability to move information securely and smoothly between the application and the database. The development team will use a Github repository to store the code and support the CI/CD process through development phases. The repository will create a code store for each member to reach and contribute to throughout the process. Our repository will utilize Copilot AI agents to ensure formatting consistency throughout the codebase. The development team will be using various linters such as ESLint (https://eslint.org/) and the standard linters that come built into the IDE being used like VSCode, JetBrains Intellij and Visual Studio.

11.2 External Libraries

Various external SDKs and APIs will be used to link to external fitness devices.

- Connect IQ (Garmin SDK): A major player in the fitness watch company has an open SDK to be used for more accurate data collection and real time updates (https://developer.garmin.com/connect-iq/overview/).
- Strava API: Strava is popular application for runners to track data. The API will allow us to connect to the strava tracking system for users of the app to help track different metrics (https://developers.strava.com/).
- Health Connect API (Android Development Health API): Android provides an API for users to be able to retrieve health data to connect across applications easier (https://developer.android.com/health-and-fitness/health-connect).

11.3 Technology and External Devices

The application will rely on the device itself for its basic activity tracker. Any additional activity tracking devices like smart/fitness watches, standalone heart rate monitors, pedometers, etc. will be integrated utilizing the intended SDK or API provided by the manufacturer.

11.4 Testing

The development team will use standard unit testing and system testing frameworks like JUnit (https://junit.org/). These tests will be manually prepared by the developers to test different features of the app. The development team will also use the frontend testing framework Detox (https://wix.github.io/Detox/docs/introduction/getting-started/). This will help to provide another layer of verification to the frontend systems.

12 Use of AI and LLMs

12.1 Code Development

AI tools will be used for front end development and to a lesser extent for back end development. Front end components will consists of screens, layouts, and UI components, of which LLMs generate well. The back end handles fitness data, virtual currency, and portfolio values, where mistakes are harder to spot, so it will be written mainly by hand, with AI used as a supporting tool.

The team will use Claude, and GitHub Copilot for code development. ChatGPT will also be used in for answering small clarifying questions and attending to tasks non-code or documentation related. Finally, GitHub Copilot will also be used to generate an overview of each pull request to help reviewers understand the changes.

12.2 Documentation Development

LLM use for documentation will be minimal. It will be limited mainly to finding and correcting grammar mistakes, along with preliminary research/brainstorming. The content of each document will be mostly written by the team. The team will continue to use class templates with the occasional help from Copilot and ChatGPT for documentation.

12.3 Verification of LLM Output

Every merge into the main branch requires manual review and approval from one team member other than the developer who wrote the branch. Reviewers check that the changes work as intended and that the code is correct, regardless of whether it was written by a person or generated by an LLM. The Copilot pull request overview helps reviewers get oriented, but does not replace either review.

13 Coding Standard

Each of the different programming languages used will follow the industry standard coding standard set by development teams today.

1. For the database, the development team will utilize a MySQL server and follow the MySQL 9.7 standards (https://dev.mysql.com/doc/refman/9.7/en/).

2. For API development, the development team will be using the Spring.io framework. This framework utilizes the Java programming language and the development will follow the the Java Oracle Code conventions standard (https://www.oracle.com/java/technologies/javase/codeconventions-programmingpractices.html).

3. For the frontend, the develpment team will utilize Typescript and will follow the Typescript style guide rules set by the typescript team (https://ts.dev/style/#identifiers).

14 Appendix — Generative AI Use Disclosure

14.1 AI Tools Used

- Copilot
- ChatGPT
- Claude

14.2 How and Where Used

Github and Copilot were used to verify and validate the development teams pull requests, as one of the 3 reviewers. It was also used to generate an overview of each pull request to help reviewers better understand the changes. Finally, it was used to generate our proprietary license in the LICENSE file.

Claude was used in the creation of the contribution table for both the Development Plan and the Problem Statement.

14.3 Verification of AI-Generated Content

The development team used AI-Generated content to aid in the process of brainstorming and idea formulation, as well as throughout the documentation process to assist with formatting. After using AI for any source of idea that would be used, the team would take the time to find a more reputable source that allowed us to verify the information and dive deeper into the topic. For formatting the development team would ensure that the documents compiled properly and looked as intended. The team was able to confirm the document through a formal pull request.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?

   A development plan allows us to think about the scope and the various different design considerations our team might have for the project. It gives us an idea of a start and end point of where we as a team agreed to go, and helps us to align on plan and expectatations before getting into the building of the project. Finally, it allows us to avoid rework later in the development process as a through plan is defined and can be easily followed.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

   Some advantages of using CI/CD are that it allows for every change to seamlessly get run against the entire test suite so that we do not introduce regressions. Changes for users get landed faster as well, without going through an entire QA process. With the seamless testing, it also allows us to catch and fix any issues right away. Some disadvantages are that it can be tricky to debug and understand at times; if your test suite/coverage is incomplete it can be difficult to implement with the main code.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

   Our only disagreement was our difference in preferred stacks for the project. We came into the project with different skills and experience, and we had to come to a compromise on what stack we would use for the project. We resolved this by having a discussion about the pros and cons of each stack, and ultimately decided on a stack that we all felt comfortable with and that would allow us to build the project effectively. Another lengthly

discussion we had, albiet not entirely a disagreement, was on branch protection, and whether or not we should have branch protection on every branch, and require reviews even for small changes made after they've previously been approved.

Appendix — Team Charter

External Goals

- Develop a finalized and working solution that we can be proud of and use as a strong project reference on resumes and in interviews.
- Develop a solution in a topic of interest that we enjoy and can relate to, so that we can maintain motivation and genuine enjoyment during development of the solution.
- Achieve a grade of at least an A (11 on McMaster's 12-point scale) in the capstone course
- Make it to the Capstone EXPO and standout positively among the other software projects
- Grow our professional skills, project management skills, and problem solving skills while working through a real world problem

Attendance

Expectations

- All team members are expected to attend all scheduled meetings, unless they have an acceptable excuse for missing the meeting
- Arrive at least 10 minutes before to the agreed upon location if the meeting is held in person, and at arrive at least 5 minutes before any scheduled virtual meetings to ensure prepardness and that the full meeting time may be utilized effectively
- If a team member is unable to attend a meeting, they are expected to notify the rest of the team in the Microsoft Teams groupchat at least 24 hours in advance, unless it is an emergency situation and out of their control
- If a team member misses a meeting or misses part of a meeting, they are expected to review the meeting notes and any relevant documents before the next meeting to get caught up on what they missed
- If a team member misses 2 meetings in a row without an acceptable excuse or proper communication with the rest of the team, the team will schedule an emergency meeting to address this issue. If the issue persists, it will be escalated to the TA and/or instructor for further action
- Arriving late to a meeting or leaving early without an acceptable excuse will result in a warning. If this happens more than 3 times, the team will schedule an emergency meeting to address this issue. If the issue persists, it will be escalated to the TA and/or instructor for further action

Acceptable Excuse

Acceptable Excuses:

- Unforseen circumstances that are out of the teammate's control (i.e. car breaks down on the way to a meeting)
- Medical appointments that were scheduled in advance and cannot be rescheduled
- Medical, personal, or family emergencies
- Personal ilness or injury that hinders the teammate's ability from attending and or providing meaningfull contributions

Unacceptable Excuses:

- Technical difficulties during virtual meetings (i.e. internet issues, computer issues, etc.). These can and should be resolved before the meeting starts
- Forgetting about a meeting or forgetting to notify the team about a prior conflict
- Other course deliverables. These meetings are scheduled in advance and thus should be planned around in order to meet deadlines for all courses

In Case of Emergency

1. Contact the rest of the team immediately about the emergency (the specific details of the emergency are not required to be shared as they may be personal)
2. Assess the situation and determine how long this emergency will hinder your progress (i.e. will just miss one meeting, will miss a deliverable deadline, will miss extended time)
3. Find a replacement/backup team member with extra capacity to cover for you
   (a) Meetings - if you have a prominent role in the meeting(s) (i.e. scribe) find and contact a team member that can fill the role of scribe for the missed meeting(s)
   (b) Deliverables - if your assigned task is yet to be completed, communicate your progress with the team so they can delegate that responsibility to another team member with extra capacity to complete the task
4. When the emergency is resolved and you are able to get back into the project, catch up on what you have missed

Accountability and Teamwork

Quality

In order to maintain consistent quality in our work, the following are expected of all team members:

- Meeting Preparation
  - Team members are expected to come to meetings fully prepared reading any necessary materials
  - Team members are expected to come to meetings with their assigned tasks completed and ready to discuss their progress with the team
- Deliverables
  - Each deliverable should be thoroughly reviewed by all team members before submission to ensure that it meets the quality standards of the team
  - Team members should strive to produce work that meets the level 4 criteria in each deliverable rubric

Attitude

- Team members are expected and encouraged to provide constructive feedback
- Team members are expected to be positive and respectful when interacting with other team members, TAs, instructors, and any other stakeholders
- Team members should be open to new ideas and willing to compromise when necessary
- Team members shuold be patient and maintain professionalism when conflict arises
- Team members should feel as though they can openly approach any other team member with questions, concerns, or ideas
- Team members should welcome all ideas for consideration, instead of shutting down ideas without discussion
- Team members should be willing to contribute their fair share of the workload

Stay on Track

- The team will leverage the Kanban board in our GitHub Documentation Project Tracker to track the status of our tasks and deliverables. Each team member is responsible for keeping their assigned issues up-to-date on the Kanban board
- The team will also leverage Avenue to Learn and the Google calendar provided by the instructor to track deadlines for deliverables
- During scheduled lecture time slots (meeting held regardless whether the lecture is running or not - meeting held after lecture if necessary), 'stand-up' style meetings will occur to monitor incremental progress to ensure that all team members stay on track to meet deliverable deadlines
- The team will leverage performance metrics on the Github repository to monitor attendance records, and contributions (i.e. commits, pull requests, etc.)
- No target contribution metrics per deliverable as workload may vary. However, by the conclusion of the project, contributions should be relatively equal among all team members ranging from 20-30% of the total contributions per team member
  - If contribution targets are not met for an individual team member by the end of the project, the team will escalate this issue to the TA and/or instructor to have the capstone grade adjusted accordingly
  - If the team notices the decline in contributions of a team member during the project, the team will adjust future workloads accordingly to balance out contributions throughout the entirety of the project and enforce fairness
- For attendance, it is expected that all team members attend at least 90% of all scheduled meetings

Team Building

- The team will engage in icebreakers and non-project related discussions for a few minutes before each meeting to better get to know each other and build meaningful relationships
- The team will get together after successfull deadline submissions and project milestones to celebrate a job well done
- The team will hold social meetings that are not project related at the start of the project, halfway through the project (end of fall term), and at the very conclusion of the project (end of winter term after the Capstone EXPO)

Decision Making

1. All ideas will be considered and discussed by the team before a decision is reached
2. The team will first attempt to come to a consensus on a decision as we should all be in agreement and aligned before making impactful decisions
3. If a consensus cannot be reached, the team will hold a vote on the decision. For an idea to be passed a majority vote of 3 out of 5 members is required
4. If the team is struggling to decide between options (i.e. they cannot clearly decide where to cast their votes), an unbiased third party will be involved to help the team come to a final decision